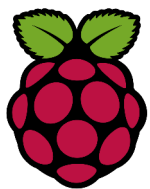


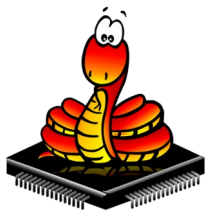


Atelier Raspi

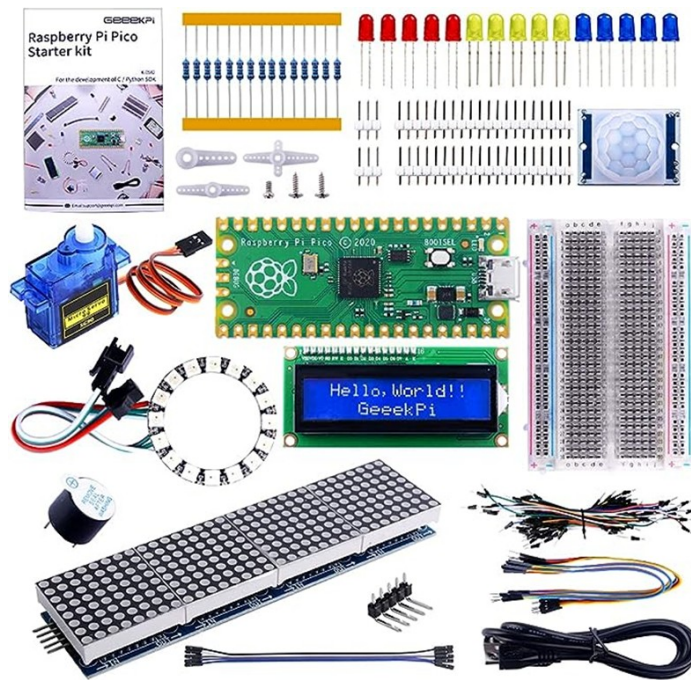
Atelier N°16 La mini serre instrumentée Module interface homme machine



Logo du Raspberry Pico



Logo du MicroPython



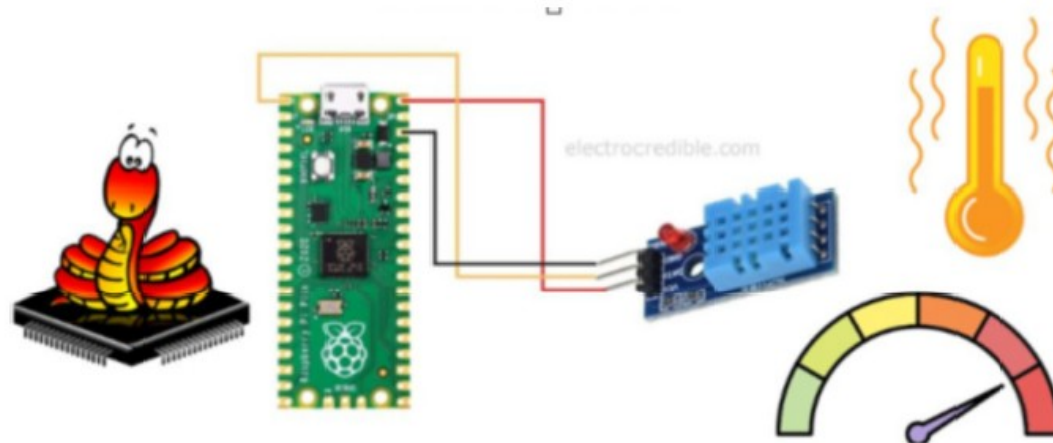
L'atelier a pour valeurs, le partage, l'aide, la formation, le faire et construire ensemble à partir de l'expérience des participants

Atelier N°16 La mini serre instrumentée

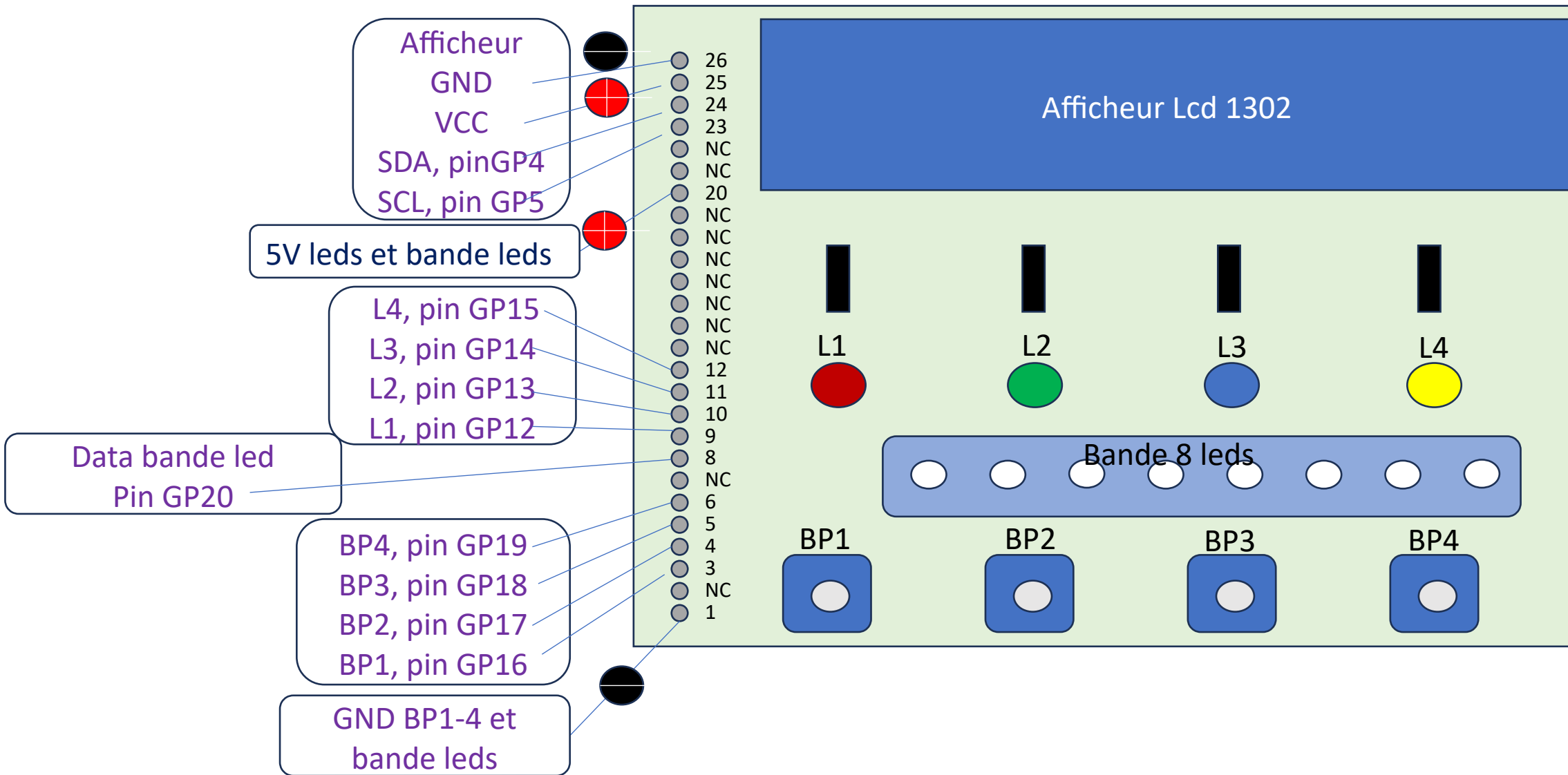
Module interface homme machine

- Etape 1 Le module interface homme machine (IHM)
- Etape 2 Le brochage des capteurs DHT11/22
- Etape 3 le câblage et placement de l'IHM
- Etape 4 les programmes utilisés
- Etape 5 le programme de la séance 1ere partie
- Etape 6 le programme de la séance 2ere partie

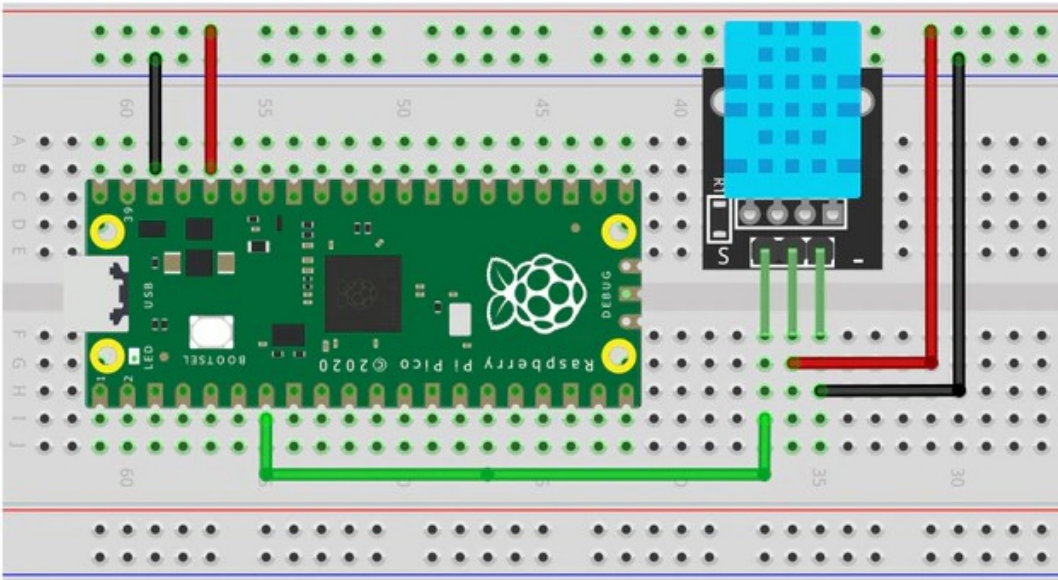
- GI1 Glossaire informatique1
- GI2 Glossaire informatique2
- GI3 Glossaire informatique3
- GI4 Glossaire informatique4
- GI5 Glossaire informatique5
- GI6 Glossaire informatique6
- GE1 Glossaire électronique



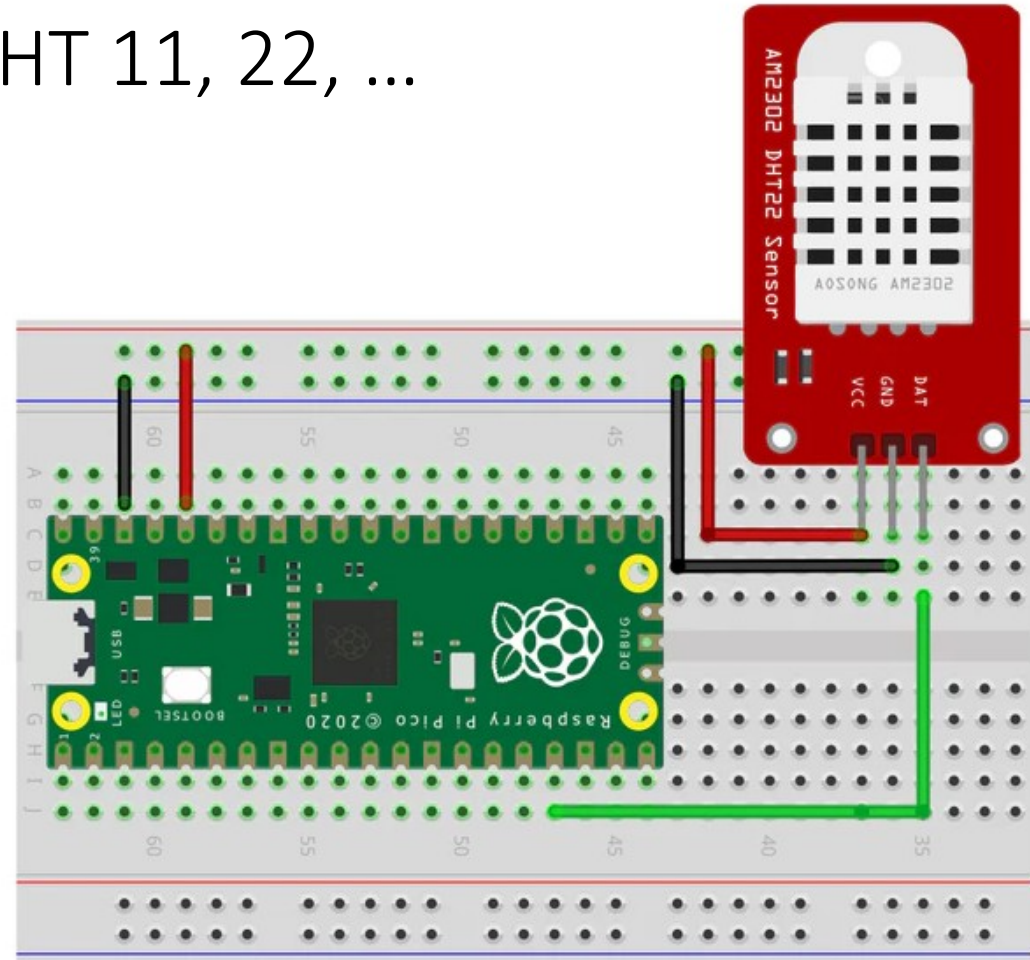
A16E1 module interface homme machine



A16E2 différents câblages du DHT 11, 22, ...



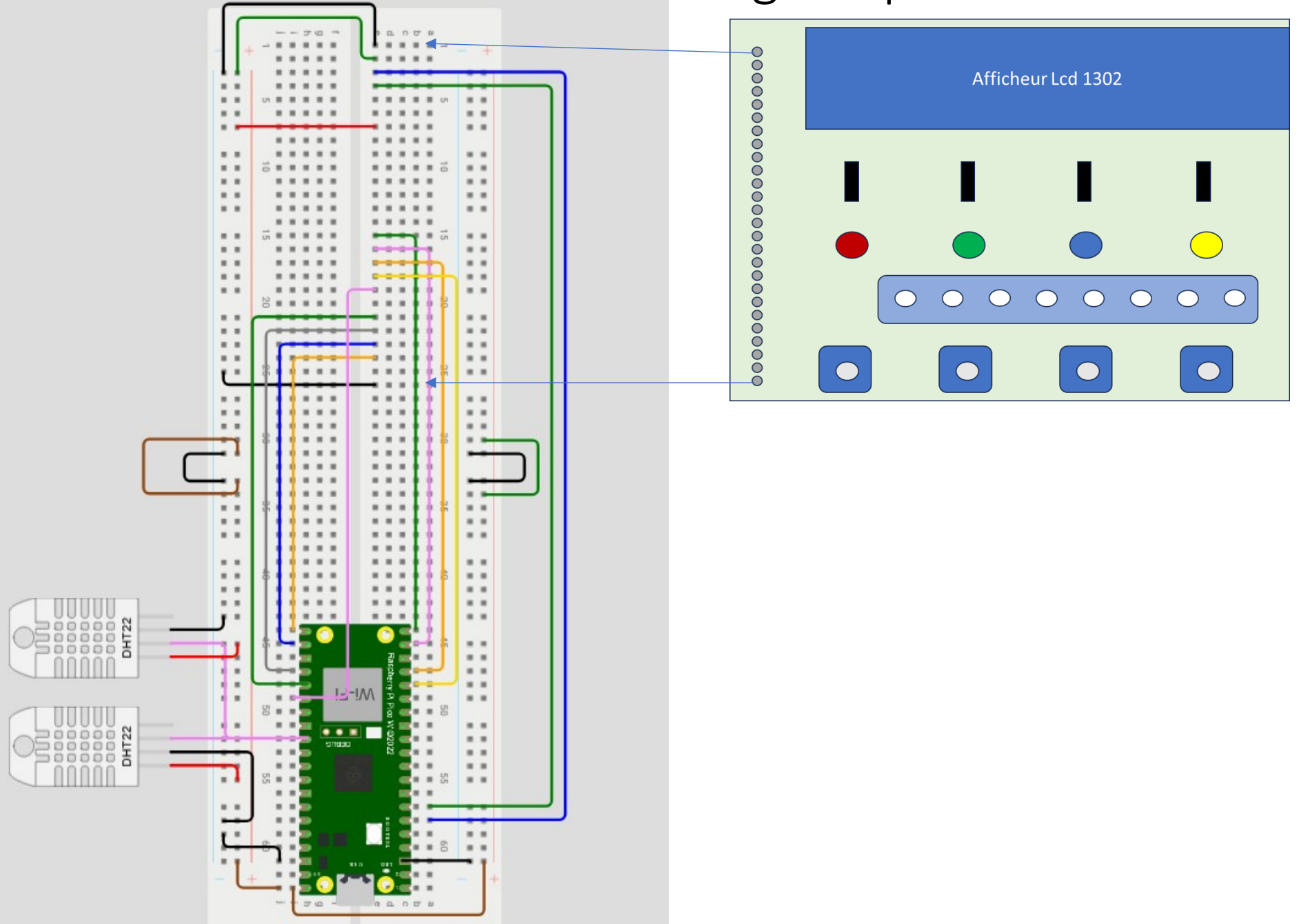
Pin de lecture du capteur: GP5



Pin GP13 **ATTENTION**

Suivant le modèle de dht22, l'alimentation + et- est inversée

A16E3 Câblage et placement de l'IHM



Attention au câblage du DHT22
Bien regarder le brochage sur
le composant

A16E4 les programmes utilisés

Au branchement de l'USB, le main.py va se lancer, il fait une séquence de test et d'initialisation de l'IHM.

Nous allons utiliser 2 librairies qui sont aussi sur le pico avant de lancer le programme d'aujourd'hui.

- lcd1602_i2c.py qui est le driver du LCD 2 lignes de 16 caractères
- quant_debounce_bp.py qui est le programme qui va détecter l'appui des boutons et qui va filtrer les rebonds sur les boutons.

Pour le programme à frapper, 260304 seance.py, l'objectif est de rentrer sur la console de Thonny 4 messages de max 16 caractères au choix du programmeur, puis le programme affiche « prêt » sur l'afficheur lcd et suivant l'appui du bouton, on a le message correspondant sur l'afficheur.

On pourra faire une évolution du programme, en fonction du message, si l'on indique une couleur, on allumera la led correspondante.

```
1 #L'objectif est de rentrer sur la console de Thonny
2 #4 messages de max 16 caractères au choix du programmeur,
3 #puis le programme affiche « prêt » sur l'afficheur lcd
4 #et suivant l'appui du bouton, on a le message correspondant
5 #sur l'afficheur.
6
7 from machine import I2C, Pin
8 import time
9 from quant_debounce_bp import Quant_Debounce_Bp
10 from lcd1602_i2c import*
11 # --- Lecture des messages dans la console ---
12 print("Attention mettre le curseur dans la console avant de taper les messages")
13 msg1 = input("Message 1 (ligne 1) : ")
14 msg2 = input("Message 2 (ligne 2) : ")
15 msg3 = input("Message 3 (ligne 1) : ")
16 msg4 = input("Message 4 (ligne 2) : ")
17 # --- LCD ---
18 i2c = I2C(0, scl=Pin(5), sda=Pin(4), freq=400000)
19 lcd = I2cLcd(i2c, DEFAULT_I2C_ADDR, 2, 16)
20 # --- Boutons ---
21 bp1 = Quant_Debounce_Bp(16, "bp1")
22 bp2 = Quant_Debounce_Bp(17, "bp2")
23 bp3 = Quant_Debounce_Bp(18, "bp3")
24 bp4 = Quant_Debounce_Bp(19, "bp4")
25 lcd.clear()
26 lcd.putstr("Pret...")
```

```
27 # --- Boucle principale ---
28 while True:
29     if bp1.get_state() == "bp1":
30         lcd.clear()
31         lcd.move_to(0,0)
32         lcd.putstr(msg1)
33         bp1.bp = None
34     if bp2.get_state() == "bp2":
35         lcd.clear()
36         lcd.move_to(0,1)
37         lcd.putstr(msg2)
38         bp2.bp = None
39     if bp3.get_state() == "bp3":
40         lcd.clear()
41         lcd.move_to(0,0)
42         lcd.putstr(msg3)
43         bp3.bp = None
44     if bp4.get_state() == "bp4":
45         lcd.clear()
46         lcd.move_to(0,1)
47         lcd.putstr(msg4)
48         bp4.bp = None
49     time.sleep(0.05)
50
```

G11 Glossaire informatique1

Instruction du langage : ne nécessite pas d'importer de module pour les utiliser

print('Bonjour') : affiche Bonjour sur la console du PC

While test : Boucle tant que test est VRAI

import : pour importer des modules contenant des fonctions

import time : permet d'appeler les fonctions relatives au temps

time.sleep(x) : arrête l'exécution du programme pendant x secondes

import machine : module contenant les fonctions permettant d'agir sur les périphériques du pi pico

from : pour simplifier l'écriture on importe certaines fonctions directement dans une variable à l'aide de « from »

from machine import Pin : permet d'accéder à la fonction Pin du module machine uniquement.

ma_led_verte = Pin(n, Pin.OUT) : permet d'indiquer comment on souhaite utiliser un GPIO,
ici on a configuré le p I/O N° n en sortie tout ou rien

ma_led_verte.Toggle() : inverse l'état de l'I/O associé à la variable.

Ici l'IO n passera de l'état haut (+3.3v) à l'état bas (0v) ou inversement

: En début de ligne permet d'écrire un commentaire, ce qui est écrit n'est pas une instruction et donc n'est pas pris en compte par le programme.

Variable **A=A+1** -> Nouvelle valeur = ancienne valeur+1 , peut s'écrire **A+=1**

print (" texte " , A), input ("texte" , B)

Opérateurs: +,-,*,/,%, **Données**: int(), float(), str(), bool()=True or False

G|2 Glossaire informatique2

Liste -> Création d'une liste, `liste_de_noms = ['Toto', 'Hubert', 'Jacques', 'Philippe']`

`liste_quelconque = ['Toto', 1, 'Philippe', 4,5,6]`

`print ('Nombre d'articles dans la liste : ', len(liste_quelconque))`

`for items in liste_quelconque:`

`print (items)` -> parcourt la liste et imprime chaque item de la liste

Pour récupérer un élément de la liste, **variable= liste[i]**, i étant l'indice d'ordre dans la liste

Dictionnaire-> liste d'éléments sous la forme d'une paire key:valeur.

`N = {'voiture':'4 roues', 'moto':'2 roues'}`

On accède à la valeur d'un key en l'utilisant comme index (clé de recherche):

`print(N['voiture'])` -> '4 roues'

Fonction-> `def nom_de_ma_fonction(args) :`

on peut ensuite l'appeler dans un programme avec: `nom_de_ma_fonction()`

Module-> `from machine import Pin` -> importe la fonction Pin du module machine

`P0 = Pin(0, Pin.OUT)` -> on paramètre le GPIO 0 en sortie

`P0.value(1)` -> on lui donne la valeur 1 ou état « haut »

G|3 Glossaire informatique 3

Boucles

for degree in range(0,180,1):

XXXXX

-> la variable degree va varier de 0 à 180 avec un pas de 1, pour chaque valeur de cette variable, XXXX sera exécuté

global ma_variable -> permet d'avoir accès à ma_variable du programme principal dans la def d'une fonction

def ma_fonction():

global ma_variable

print(ma_variable)

ma_variable += 2

return ma_variable

Return ma_variable -> permet de donner au reste du programme la nouvelle valeur de ma_variable ou d'une nouvelle variable si créée dans la fonction

G|4 Les fonctions du module time

time.time() -> donne le temps présent en ms

time.time_diff(ticks1, ticks2) -> donne la différence entre ticks1 et 2

time.sleep (2) -> met un délai de 2 secondes

utime travaille comme time mais en ms

utime.sleep (100) -> met un délai de 100ms

G|5 Glossaire informatique 3

Les tests:

l'instruction **if...else** permet d'exécuter un bloc de code en fonction de la véracité d'une condition,

où le corps de l'instruction **if** s'exécute si la condition est **vraie**,

et le corps de l'instruction **else** s'exécute si elle est **fausse**,

avec une indentation pour définir les blocs de code.

Dans la suite des instructions du programme,

Exemple:

Dans le programme A=2

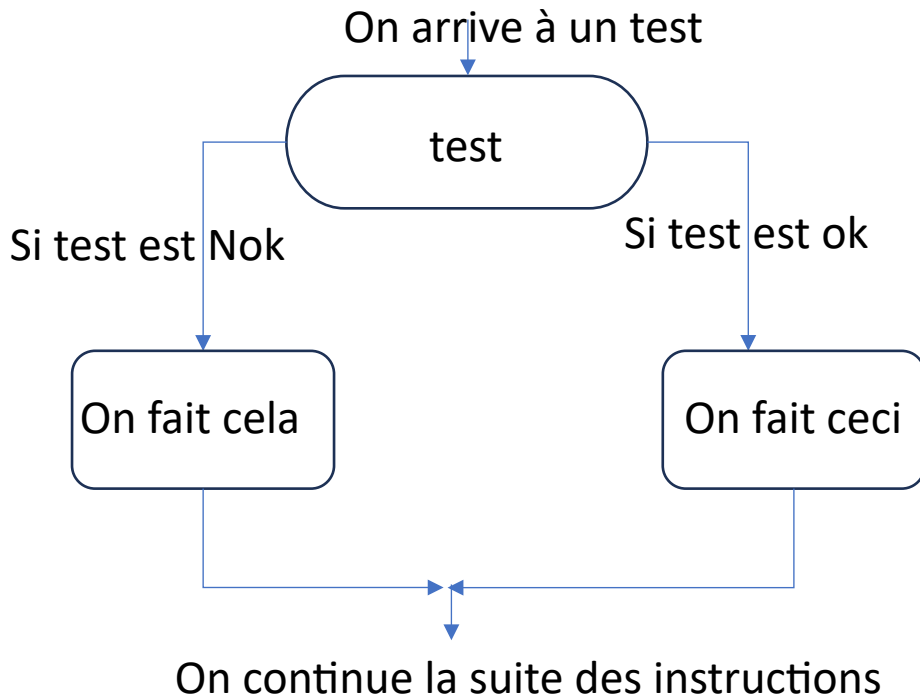
If A > 10:

print('A supérieur à 10')

else:

print ('A inférieur à 10')

On continue d'exécuter les instructions qui suivent



Gl6 Les fonctions du module PWM

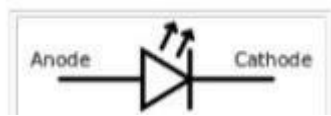
Fonctions utilisées:

```
servoPinPan = PWM(Pin(16))          # servoPinPan est un objet, initialise le Pin GPIO 16 en sortie PWM  
servoPinPan.freq(50)                # Initialise la fréquence des signaux carrés à 50Hz soit 20ms  
servoPinPan.duty_u16(int(newDuty))  # duty_u16 est le rapport cyclique entre le temps du signal carré haut sur  
sa période, new duty est une variable, le max est à 65535.
```

```
servoPinPan.deinit()                # réinitialise le PWM
```

```
servoX.duty_ns(pulseDX) #commande le rapport cyclique du PWM en nanosecondes suivant la variable  
pulseDX de l'objet servoX
```

GE1 Glossaire électronique



Symbole de la diode LED



Pile



Résistance



BP

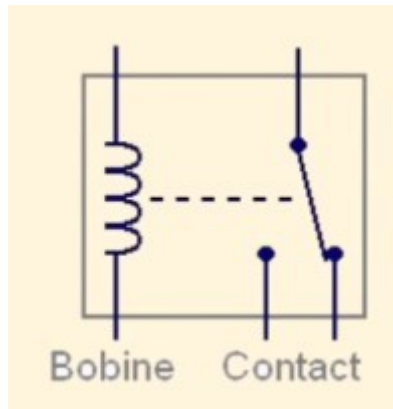
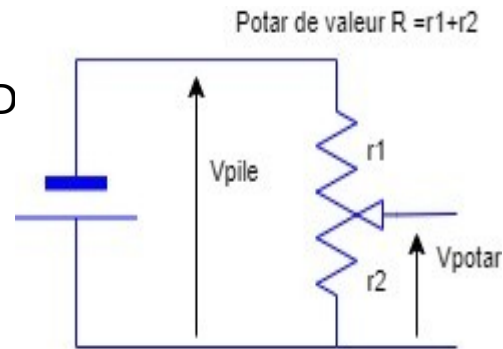


Schéma électrique du relais



$$V_{potar} = V_{pile} * r2 / (r1 + r2)$$



Condensateur
de filtrage signal



Condensateur
Chimique, réserve d'énergie

Couleurs	Tension de seuil ou Vf	If (mA)	Longueur d' onde
Rouge	1,6 V à 2 V	6à20	650 à 660 nm
Jaune	1,8 V à 2 V	6à20	565 à 570 nm
Vert	1,8 V à 2 V	6à20	585 à 590 nm
Bleu	2,7 V à 3,2 V	6à20	470 nm
blanc	3,5 v à 3,8 v	30	

